PROYECTO INTERMODULAR · CURSO 2026-2027

VulnFollowUp

Guía paso a paso
para Alex, Adri y Pablo

CONSTRUIR, REVISAR Y SABER EXPLICARLO

Pasos sencillos, ejemplos y reparto rotativo para que los tres entendáis el proyecto completo.

En cada tarea: una persona la lleva, otra la revisa y otra la demuestra. Los papeles cambian durante el proyecto.

Base de trabajo: enunciado académico de 23 páginas y requisitos de los diez módulos.

29 de octubre de 2026 · Versión 2.0

Las propuestas de esta guía se distinguen de los requisitos oficiales. Los resultados y evidencias se completan durante el desarrollo real.

Índice de trabajo

Empezad juntos por los apartados 1-4. Después seguid los pasos, probad el resultado y explicadlo entre compañeros. El índice enlaza con las secciones.

1. Empezad aquí: qué vais a construir . . . 3

2. Qué os piden y qué podéis dejar para después . . . 4

3. Las palabras técnicas traducidas a algo entendible . . . 6

4. Reparto entre Alex, Adri y Pablo sin trabajar aislados . . . 7

5. Primer paso: preparar el ordenador y arrancar algo . . . 11

6. Segundo paso: decidir qué información vais a guardar . . . 12

7. Tercer paso: login y primeras operaciones de la API . . . 13

8. Cuarto paso: guardar bien la gravedad CVSS . . . 16

9. Quinto paso: importar veinte registros sin dejar datos a medias . . . 17

10. Sexto paso: seguir la resolución del problema . . . 18

11. Séptimo paso: que dos usuarios no se pisen los cambios . . . 19

12. Octavo paso: generar un informe sin bloquear la aplicación . . . 20

13. Noveno paso: relacionar proveedores con sus riesgos . . . 21

14. Décimo paso: construir la web que usará la persona . . . 22

15. Undécimo paso: consultar y actualizar desde Android . . . 23

16. Duodécimo paso: empaquetar, integrar y probar todo . . . 24

17. Los trabajos de cada módulo, explicados por entregable . . . 26

18. Preparar una memoria única que los tres entendáis . . . 28

19. Prepararos para preguntas a cualquiera de los tres . . . 29

20. Lista de entrega y vuestra primera sesión . . . 31

21. Fuentes y qué debéis contrastar . . . 33

1. Empezad aquí: qué vais a construir

Equipo: Alex, Adri y Pablo. Proyecto intermodular de 2.º DAM, curso 2026-2027. Versión 2 de la guía, revisada el 29 de octubre de 2026.

Vais a construir una aplicación que permite apuntar problemas de seguridad de una empresa, ver cuáles son más graves y controlar quién los está solucionando. La misma información se podrá consultar desde una web y desde una aplicación Android.

Pensad en una lista de tareas de mantenimiento, pero con más información: qué equipo tiene el problema, qué gravedad tiene, quién lo arregla, en qué estado está y qué proveedor está relacionado. No tenéis que descubrir problemas atacando equipos: trabajaréis con ejemplos inventados o registros públicos documentados.

1.1. Un ejemplo que usaremos durante toda la guía

La empresa tiene un servicio llamado **API de pagos**. Ese servicio lo presta un proveedor ficticio llamado **HostingSur**. Registráis una vulnerabilidad inventada, **VFU-0001**, que afecta a la API de pagos. Su valoración CVSS es 9,8 y su severidad es Crítica.

Un usuario ANALYST asigna el trabajo a un responsable, añade una fecha objetivo y pasa la vulnerabilidad a «Pendiente de remediación». Después añade la evidencia de la corrección y la pasa a «Resuelta». El móvil consulta el mismo registro y muestra el cambio. El informe de HostingSur permite ver si todavía le quedan problemas pendientes.

Al terminar el proyecto, los tres tenéis que saber hacer y explicar ese recorrido. La guía reparte quién toma la iniciativa en cada tarea, pero el conocimiento y la entrega son compartidos.

1.2. Cómo leer la guía sin agobiaros

1. Leed los apartados 1-4 juntos: explican el proyecto y el reparto.
2. Realizad las tareas de los apartados 5-15 en el orden del tablero del apartado 4.
3. Al terminar cada tarea, haced su comprobación y la explicación entre compañeros.
4. Guardad pruebas y documentación mientras trabajáis. Los apartados 16-18 explican cómo.
5. Reservad tiempo para ensayar la defensa del apartado 19.
6. Antes de entregar, repasad la lista del apartado 20.

No hace falta entender todas las palabras técnicas el primer día. El apartado 3 las traduce a lenguaje sencillo. Si una tarea está bloqueada, buscad cuál es su paso anterior; no intentéis terminar tres pantallas que dependen de un servicio que todavía no existe.

1.3. Qué significan las etiquetas

OBLIGATORIO: lo exige el enunciado. **PROPUESTA:** una forma recomendada de organizaros o programarlo; podéis cambiarla manteniendo el requisito. **CONSULTAR:** hay una ambigüedad que conviene aclarar con el profesor. El reparto de nombres es una propuesta inicial: no supone que conozcamos el nivel o las preferencias de cada uno.

La fuente principal es el PDF de 23 páginas `Proyecto_Intermodular_2DAM_VulnFollowUp (1).pdf`. Las referencias a páginas corresponden al enunciado. Esta guía os ayuda a realizar el proyecto; no sustituye la memoria final ni demuestra que el programa ya funcione.

2. Qué os piden y qué podéis dejar para después

2.1. Las cuatro piezas del proyecto

Pieza	Explicado fácil	Carpeta
Base de datos	El almacén donde queda guardada la información	`database/`
Backend Java	El programa que recibe peticiones, comprueba permisos y guarda o consulta datos	`backend/`
Cliente web o escritorio	Las pantallas que usa la persona desde el ordenador	`desktop-client/`
Aplicación Android	Las pantallas para consultar y seguir el trabajo desde el móvil	`android-app/`

Los proveedores no son una quinta aplicación independiente: su gestión se añade al mismo backend, la misma base de datos y las pantallas necesarias.

2.2. Lista de mínimos, en palabras normales

- Entrar con usuario y contraseña. Tener tres tipos de usuario: ADMIN, ANALYST y VIEWER.
- Registrar activos: por ejemplo, un servidor, una base de datos o una API de pagos.
- Registrar vulnerabilidades y relacionarlas con uno o varios activos.
- Guardar versión CVSS, vector, puntuación, severidad y fecha de valoración, comprobando que coincidan.
- Buscar y filtrar por severidad, estado, activo, responsable y rango de puntuación.
- Cambiar el estado: Abierta, Pendiente de remediación, Resuelta y Cerrada. Guardar responsable, fecha objetivo y evidencia del trabajo.
- Cargar activos y proveedores iniciales e importar un JSON con al menos 20 vulnerabilidades.
- Si falla una vulnerabilidad del lote, evitar que se guarde media importación.
- Demostrar qué pasa si dos personas editan a la vez y cómo evitáis perder cambios.
- Ejecutar una tarea en segundo plano y lanzar un proceso externo que genere un informe.
- Mostrar un dashboard con cantidades por severidad y estado.
- Registrar proveedores TIC, relacionarlos con activos y consultar sus problemas pendientes.
- Tener una web o cliente de escritorio distribuible y una app Android con consulta básica sin conexión.
- Preparar las pruebas, la documentación de los diez módulos y una defensa de 20 minutos más preguntas.

OBLIGATORIO, páginas 3-19: ambos clientes usan el mismo backend. No vale que la web lea unos datos y Android muestre una lista inventada guardada en su código.

2.3. Lo que no hace falta añadir al principio

No os piden escanear redes, explotar vulnerabilidades, cobrar con tarjeta, usar datos personales reales o publicar el servicio en Internet. Dejad notificaciones, integración con tickets, sincronización de cambios sin conexión y automatización de fuentes externas para después de completar los mínimos.

La referencia a DORA aporta el contexto profesional. No convierte vuestra aplicación en una certificación de cumplimiento legal.

2.4. Dos consultas al profesorado

CONSULTAR - Informes de DI: la página 11 habla de exportación PDF/CSV y la 23 sitúa PDF entre las ampliaciones. El propio documento lo señala en el comentario CL1. Preparad CSV y gráficos básicos interactivos; preguntad si el PDF es obligatorio. No deis DI por cerrado sin aclararlo o sin cubrir la interpretación más exigente.

CONSULTAR - Ampliación de Inglés: el comentario CL2 de la página 14 señala dudas sobre su evaluación oral y algunos entregables. Preparad README y API en inglés, glosario, resumen ejecutivo y recursos para traducir la interfaz; preguntad cómo se evaluarán.

PROPUESTA - CVSS: empezar con CVSS 3.1 Base. El enunciado permite 3.1 o 4.0; no exige desarrollar las dos. Documentad vuestra elección y rechazad versiones que no soportéis.

3. Las palabras técnicas traducidas a algo entendible

3.1. Diccionario de supervivencia

Palabra	Qué significa aquí	Ejemplo
Activo	Algo de la empresa que queréis proteger	API de pagos
Vulnerabilidad	Un problema de seguridad que registráis	VFU-0001 afecta a la API de pagos
Proveedor TIC	Empresa que presta un servicio tecnológico	HostingSur
Backend	Programa central con las reglas y los permisos	Comprueba si podéis cambiar el estado
API	Forma acordada de hablar con el backend	La web pide la lista de vulnerabilidades
Endpoint	Dirección concreta de una operación de la API	`GET /api/assets` consulta activos
REST y HTTP	Convenciones para pedir y enviar información	GET consulta; POST crea
JSON	Texto estructurado para intercambiar datos	Un registro con título, estado y puntuación
CRUD	Crear, consultar, modificar y borrar o desactivar	Dar de alta y editar un activo
DTO	Objeto con los datos que una operación permite recibir o devolver	Formulario de alta sin campos internos de seguridad
Token	Credencial temporal que el servidor comprueba tras el login	Permite hacer peticiones sin reenviar la contraseña
Hash	Transformación usada para verificar contraseñas sin guardarlas legibles	El servidor conserva el hash, no la contraseña original
Transacción	Grupo de cambios que se guarda completo o se deshace	Importar veinte registros como un único lote
Rollback	Deshacer los cambios de una transacción que falla	No conservar once registros si el número doce falla
Concurrencia	Varias operaciones avanzan a la vez	Dos usuarios editan la misma vulnerabilidad
Caché	Copia local de datos consultados	Android muestra el último listado sin conexión
Dashboard	Pantalla de resumen con cifras y gráficos	Cuántas críticas siguen abiertas
Desplegar o empaquetar	Preparar el programa para que otro lo ejecute	SPA servida por Nginx en Docker
Evidencia	Algo verificable que demuestra un resultado	Una prueba, un CSV o una captura explicada

3.2. Qué pasa cuando pulsáis Guardar

Ejemplo: Adri cambia el estado desde la web.

1. La pantalla recoge el nuevo estado y la evidencia escrita.
2. La web envía una petición HTTP a la API.
3. El backend comprueba quién es Adri y si su usuario puede modificar.
4. Comprueba que los datos son válidos y que nadie ha cambiado antes ese registro sin que Adri lo sepa.
5. Guarda el cambio y su auditoría en la base de datos.
6. Devuelve una respuesta. Solo entonces la pantalla anuncia que está guardado.
7. Pablo refresca el detalle en Android y ve el dato actualizado.

La web y el móvil no deciden solos qué puede guardarse. La última palabra la tiene el backend. La base de datos no debe ser accesible directamente desde los clientes.

3.3. Tecnologías propuestas para no elegir veinte cosas

PROPUESTA: Java 21 y Spring Boot para la API; PostgreSQL como base de datos; React con TypeScript y Vite para la web; Kotlin con Compose y Room para Android; Docker con Nginx para distribuir la web. Maven Wrapper y Gradle Wrapper ayudan a usar una configuración de construcción compartida.

Spring Boot ayuda a crear servicios Java; JPA/Hibernate ayuda a guardar objetos en tablas; React construye pantallas por componentes; Room guarda datos estructurados en Android. No hace falta dominar todo antes de empezar: aprended cada herramienta con la tarea pequeña que la necesita.

Podéis cambiar la propuesta si ya habéis trabajado con otras tecnologías admitidas por el enunciado. Elegid una sola combinación y registrad las versiones concretas. No uséis sin comprobar el Node 18 del README inicial para un Vite actual: la documentación consultada exige versiones posteriores. Fuentes técnicas de apoyo: [R2], [R3] y [R4].

Comprobación entre los tres: cada uno explica el recorrido de Guardar utilizando las palabras pantalla, API, permiso, validación, base de datos y respuesta. Si hay una palabra que no entendéis, aclaradla antes de programar su parte.

4. Reparto entre Alex, Adri y Pablo sin trabajar aislados

4.1. La regla principal del equipo

Cada tarea tiene tres papeles: quien la lleva, quien la revisa y quien la demuestra. Los papeles van cambiando. Así cada uno tiene trabajo propio y los tres recorren todo el sistema.

- **Lleva:** prepara la solución y realiza la mayor parte del trabajo de esa tarea. Explica sus decisiones y avisa si se bloquea.
- **Revisa:** lee lo que se ha hecho, comprueba cómo conecta con el resto y aporta una prueba, una corrección o una mejora concreta. No basta con decir «lo veo bien».

- **Demuestra:** arranca esa parte en su equipo o entorno de pruebas, realiza el recorrido y lo explica con sus palabras sin que quien la programó le vaya dictando.

Los tres pueden programar. Los papeles no significan que uno escriba siempre código y otro solo haga capturas. En cada grupo de tareas cambia quién lleva, revisa y demuestra; también se alterna a qué compañero se revisa.

4.2. Tablero de tareas técnicas

PROPUESTA inicial: se reparte sin asumir quién sabe más. Cada uno lidera seis tareas, revisa seis y demuestra seis. Eso no garantiza el mismo esfuerzo: revisad las horas reales y redistribuid si una tarea pesa mucho más.

Tarea	Lleva	Revisa	Demuestra
T01. Acordar alcance, tecnologías y contrato inicial	Alex	Adri	Pablo
T02. Crear proyectos y hacer que arranquen	Adri	Pablo	Alex
T03. Diseñar tablas y preparar datos de prueba	Pablo	Alex	Adri
T04. Login y permisos de los tres perfiles	Alex	Pablo	Adri
T05. API de alta, consulta y edición de activos	Adri	Alex	Pablo
T06. API de vulnerabilidades y relación con activos	Pablo	Adri	Alex
T07. Validación CVSS y clasificación de severidad	Alex	Adri	Pablo
T08. Importación JSON, errores y rollback	Adri	Pablo	Alex
T09. Remediación, responsables, evidencia y auditoría	Pablo	Alex	Adri
T10. Prueba de edición simultánea y protección	Alex	Pablo	Adri
T11. Tarea en segundo plano y proceso de informe	Adri	Alex	Pablo
T12. Proveedores, regla de riesgo e informe	Pablo	Adri	Alex
T13. Web: login, formularios y detalle	Alex	Adri	Pablo
T14. Web: filtros, dashboard y exportación	Adri	Pablo	Alex
T15. Android: login, lista, filtros y detalle	Pablo	Alex	Adri
T16. Android: cambio de estado y consulta sin conexión	Alex	Pablo	Adri
T17. Distribución web y preparación del APK	Adri	Alex	Pablo
T18. Pruebas integradas y recorrido completo	Pablo	Adri	Alex

Ejemplo real de reparto: Alex lleva el login de la API, pero Pablo lo revisa enviando una operación prohibida con VIEWER y Adri lo arranca y explica. Más tarde Pablo lleva Android, Alex revisa su conexión y Adri lo demuestra. Cada uno tiene responsabilidades en varias partes del proyecto.

4.3. Orden de las tareas y dependencias

1. **T01-T03:** decidir qué haréis, arrancar proyectos y crear el almacén de datos.
2. **T04-T06:** conseguir login y un registro de vulnerabilidad real desde la API.
3. **T07-T09:** validar CVSS, importar datos y completar la remediación.
4. **T10-T12:** resolver concurrencia, proceso de informe y proveedores.
5. **T13-T16:** completar web y Android conectados al servicio real.
6. **T17-T18:** distribuir, integrar y repetir el proyecto desde cero.

No es obligatorio esperar a T13 para abrir el frontend: después de T06, haced juntos una primera consulta sencilla desde web y Android. Es una conexión mínima para descubrir problemas pronto. Luego cada pantalla se completa cuando su operación API funciona. Los wireframes y los textos pueden prepararse antes; no inventéis respuestas definitivas distintas para cada cliente.

Qué puede avanzar en paralelo: con el contrato acordado, uno prepara datos, otro construye un formulario y otro revisa la API, siempre indicando qué depende de qué. Si una función aún no existe, dejad claro que su respuesta de prueba es temporal y sustituidla antes de cerrar la tarea.

4.4. Documentación repartida de la misma forma

Trabajo	Lleva	Revisa	Demuestra o defiende
Inglés escrito: README, API, glosario y resumen	Alex	Adri	Pablo
Inglés oral: guion, vídeo y bloque de defensa	Adri	Pablo	Alex
IPE: negocio, costes, planificación y actas	Pablo	Alex	Adri
Digitalización: diagnóstico y flujo del dato	Alex	Pablo	Adri
Sostenibilidad: mediciones, informe y decálogo	Adri	Alex	Pablo
Memoria común, anexos y declaración de IA	Pablo	Adri	Alex

Quien lleva un documento coordina su redacción y coherencia. Los datos, decisiones y reflexiones los aportan los tres. Los tres hablan en el vídeo, ensayan la defensa, revisan su contribución individual y actualizan sus portfolios.

4.5. Rutina de trabajo de cada sesión

Al empezar, cinco minutos juntos: cada uno dice qué terminó, qué hará y qué necesita de otro. Revisad una sola lista de pendientes. Un bloqueo de API que afecta a dos clientes va primero que un cambio de color.

Mientras trabajáis: tareas pequeñas que puedan enseñarse al final de una sesión; archivos y cambios identificados; decisiones anotadas. Si dos personas van a tocar el mismo archivo, coordinadlo. Si compartís cambios con Git, revisadlos antes de integrarlos y no dejéis toda la integración para el final.

Al cerrar una tarea, unos quince minutos como referencia: quien la lleva explica el cambio; quien revisa prueba un caso normal y uno que falla; quien demuestra repite el recorrido y contesta preguntas. Si no cabe en ese tiempo porque es compleja, divididla en tareas más pequeñas o dedicad otra revisión.

Ese tiempo también es trabajo del equipo. No añadáis reuniones de una hora para cada pequeño botón ni obliguéis a los tres a programar exactamente lo mismo.

4.6. Cuándo podéis poner una tarea en Terminada

[ ] Funciona el caso normal y se ha probado un error importante.

[ ] Quien revisa ha dejado una aportación o un resultado verificable.

[ ] Otra persona puede ejecutarla siguiendo instrucciones, sin ayuda constante del autor.

[ ] Los tres pueden explicar para qué sirve, dónde está y con qué partes conecta.

[ ] Hay una captura, prueba, salida o documento que lo demuestra.

[ ] Se han actualizado las instrucciones si ha cambiado el arranque o la API.

Si cumple solo «a mí me funciona», sigue en Revisión, no en Terminada.

4.7. Cómo comprobar que los tres sabéis el proyecto

Tenéis una plantilla preparada en `docs/equipo/CONOCIMIENTO.md`: filas con los temas del proyecto y columnas Alex, Adri y Pablo. Todas las casillas empiezan como **Pendiente**. El reparto y la ficha de tarea están también en `docs/equipo/TABLERO_COMPARTIDO.md`, para que podáis actualizar el trabajo sin editar esta guía.

Usad cuatro niveles: **Entiendo** si lo podéis explicar; **Ejecuto** si lo arrancáis y probáis; **Defiendo** si además respondéis por qué se hizo así y qué pasa si falla. No marquéis Defiendo porque hayáis visto una demostración de un compañero.

Al cerrar cada bloque, sacad preguntas al azar durante diez minutos. Cada uno responde a temas que no ha liderado. Antes de la entrega, haced también un ensayo individual: los tres contestan por separado cinco preguntas, sin ayudarse, y después comparáis. Así detectáis si solo uno sabe responder a concurrencia o a Android.

No hace falta memorizar cada línea. Sí saber localizar las clases principales, seguir un dato de pantalla a base de datos, explicar las decisiones y repetir las pruebas importantes.

4.8. Tiempo: una orientación, no una promesa

El enunciado indica unas **50 horas de equipo para el núcleo técnico** y sitúa los trabajos transversales en las horas de cada módulo. No asumáis que son 50 por persona. Acordad cómo se cuentan y anotad horas reales de construcción, revisión e integración.

Como referencia: 5 horas para alcance y entorno; 4 para modelo; 6 para login y API básica; 4 para CVSS e importación; 3 para concurrencia; 3 para proceso de informe; 4 para proveedores; 6 para web; 5 para Android; 6 para pruebas y distribución; 4 para revisión técnica y ensayo. Total: 50 horas de equipo.

Es un presupuesto ajustado para gente que ya conoce las herramientas. El aprendizaje, las explicaciones compartidas o un bloqueo pueden requerir más. Revisad la previsión tras T03 y T09; consultad el calendario y recortad ampliaciones antes de recortar pruebas o comprensión. No atribuyáis todas las horas de un bloque solo a quien lo lidera.

5. Primer paso: preparar el ordenador y arrancar algo

T01: lleva Alex, revisa Adri y demuestra Pablo. T02: lleva Adri, revisa Pablo y demuestra Alex.

5.1. Qué queréis conseguir

Que los tres tengáis la misma combinación de herramientas y podáis arrancar un backend, una pantalla web y una pantalla Android. Todavía no tienen que hacer todo: solo demostrar que el entorno está preparado.

La revisión inicial del repositorio encontró carpetas y README, pero no aplicaciones implementadas. Por eso no basta con ejecutar las instrucciones antiguas del README: todavía faltan proyectos y archivos de construcción. Comprobad el estado actual antes de crear algo que ya exista.

5.2. Qué hacéis, en orden

1. Crear `docs/decisiones_tecnicas.md`. Escribir tecnologías, versiones, puerto del backend, forma de autenticación, versión CVSS y forma de distribuir la web.
2. Instalar o comprobar JDK, Node compatible, Android Studio y PostgreSQL. Apuntar las versiones realmente utilizadas.
3. Crear el proyecto Spring Boot en `backend/` con Maven y dependencias de web, validación, seguridad, JPA y PostgreSQL. Conservar el wrapper de Maven.
4. Configurar una base de datos de desarrollo y lograr que el backend conecte con ella.
5. Crear un endpoint de salud sencillo: al consultarlo debe indicar que el servicio está arrancado, sin mostrar secretos.
6. Crear React/TypeScript en `desktop-client/`. Como ya hay un README, no aceptéis sobrescrituras sin revisar: podéis generar en una carpeta temporal y trasladar los archivos necesarios.
7. Crear el proyecto Android y comprobar una pantalla en el emulador.
8. Preparar configuración de ejemplo y documentar cómo aporta cada uno sus valores locales. No guardar secretos reales en el repositorio.
9. Actualizar las instrucciones de arranque con lo que acaba de funcionar.

5.3. Un fallo habitual explicado

El archivo `.env` no se lee mágicamente en todos los programas. Decidid cómo pasar los valores al backend: configuración del IDE, variables del sistema o configuración de Docker. El README debe decir cuál habéis usado.

También hay una inconsistencia de nombres inicial: un README cita `data_initial.sql` y otro `seed_data.sql`. Elegid uno; en esta guía usamos `database/seed_data.sql`.

Lo comprobáis así: Alex sigue las instrucciones de Adri desde su equipo; Pablo comprueba que las versiones y los pasos estén completos. Después intercambiáis la explicación. Guardad una captura del servicio y de las dos pantallas iniciales.

Los tres debéis responder: ¿qué hay que arrancar primero?, ¿dónde cambia la conexión a la base de datos?, ¿qué archivo permite construir el backend con la configuración compartida?

6. Segundo paso: decidir qué información vais a guardar

T03: lleva Pablo, revisa Alex y demuestra Adri.

6.1. Antes de dibujar tablas, contad la historia

Necesitáis usuarios que trabajan con activos. Los activos tienen proveedores. Las vulnerabilidades afectan a activos, tienen una valoración CVSS y van cambiando de estado. Los comentarios explican qué se hizo y la auditoría guarda quién realizó cambios importantes.

Una tabla es una colección de elementos del mismo tipo. Una fila es un elemento. Un identificador permite reconocerlo. Una clave foránea relaciona una fila con otra. Ejemplo: el activo API de pagos tiene una referencia al proveedor HostingSur.

6.2. Modelo mínimo propuesto

Tabla	Qué guarda	Ejemplo o detalle
`roles`	Los tres perfiles permitidos	ADMIN, ANALYST, VIEWER
`users`	Usuario, hash de contraseña, rol y estado activo	Usuario de pruebas analyst
`providers`	Nombre, servicio, criticidad, contrato, revisión y contacto	HostingSur; contacto ficticio
`assets`	Nombre, tipo, responsable, entorno, estado y proveedor	API de pagos
`vulnerabilities`	Identificador, título, descripción, referencias, fecha, estado, responsable, fecha objetivo y versión de edición	VFU-0001
`vulnerability_assets`	Qué vulnerabilidad afecta a qué activo	VFU-0001 afecta a API de pagos y servidor
`cvss_assessments`	Versión CVSS, vector, score, severidad, fecha y autor	3.1; 9,8; Crítica
`comments`	Autor, fecha, comentario y evidencia breve	Se aplicó la corrección en pruebas
`audit_events`	Quién cambió qué y cuándo	Cambio de Abierta a Pendiente
`jobs`	Seguimiento de importaciones o informes	Trabajo 15: completado o fallido

La tabla `jobs` es una propuesta para seguir tareas. Para simplificar, se propone una valoración CVSS actual por vulnerabilidad y un proveedor por activo. Si queréis varios proveedores por activo o varios contratos por proveedor, habrá que ampliar el modelo; explicadlo antes de tocar las pantallas.

6.3. La relación que os preguntarán seguro

Una vulnerabilidad puede afectar a varios activos y un activo puede tener varias vulnerabilidades. Eso se llama **muchos a muchos**, o N:M. Necesitáis una tabla intermedia, `vulnerability_assets`.

Ejemplo: VFU-0001 afecta al activo 1 y al 2. Guardáis dos asociaciones. No escribáis «1,2» dentro de una columna de texto. La pareja vulnerabilidad-activo no debe repetirse.

6.4. Qué hacéis, en orden

1. Dibujar las tablas y relaciones en un diagrama entidad-relación.
2. Elegir campos obligatorios y opcionales. El identificador y el usuario no se repiten.
3. Elegir códigos internos de estado, por ejemplo `PENDIENTE_REMEDIACION`; la pantalla puede mostrar el texto completo con espacios.
4. Guardar el score CVSS como decimal de una cifra, por ejemplo `DECIMAL(3,1)`, con rango 0-10.
5. Crear el esquema y las restricciones en SQL. Añadir claves y relaciones, no solo columnas.
6. Crear datos iniciales con cuentas de prueba, proveedores y activos.
7. Preparar consultas para filtros, dashboard e informe de proveedores.
8. Probar todo en una base vacía y anotar la ejecución.

Si usáis migraciones, son los cambios numerados que construyen el esquema. Mantenedlas coherentes con los scripts de entrega; no dejéis al ORM borrar y recrear tablas por sorpresa. Desactivad activos para conservar el historial en lugar de borrar referencias necesarias.

Lo comprobáis así: Adri crea la base siguiendo los pasos de Pablo; Alex prueba una relación inválida y una asociación duplicada. Deben rechazarse. Después los tres señalan dónde se guarda cada dato del ejemplo VFU-0001.

Pregunta clave: si la misma vulnerabilidad afecta a dos activos de HostingSur, ¿el informe cuenta una vulnerabilidad o dos? Debe contar una vulnerabilidad con dos activos afectados, no inflar el total.

7. Tercer paso: login y primeras operaciones de la API

T04: lleva Alex, revisa Pablo y demuestra Adri. T05: lleva Adri, revisa Alex y demuestra Pablo. T06: lleva Pablo, revisa Adri y demuestra Alex.

7.1. Qué queréis conseguir

Entrar con una cuenta real de pruebas y crear un activo y una vulnerabilidad mediante la API. Antes de construir todos los formularios, comprobad estas operaciones con una herramienta de peticiones HTTP o con pruebas automáticas.

7.2. Primero el login y los permisos

1. Crear usuarios de prueba con contraseñas protegidas por hash con salt; nunca guardarlas legibles en la tabla.
2. Implementar login con Spring Security y token o sesión. Propuesta: token temporal validado por el servidor.
3. Devolver al cliente solo datos públicos del usuario y la credencial necesaria; nunca el hash.
4. Decidir qué operaciones permite cada rol.
5. Probar una credencial válida, una contraseña errónea, una petición sin login y una escritura con VIEWER.

6. Mostrar errores comprensibles y evitar contraseñas y tokens completos en logs.

Operación	ADMIN	ANALYST	VIEWER
Consultar activos, vulnerabilidades y resumen	Sí	Sí	Sí
Crear o modificar activos y vulnerabilidades	Sí, propuesto	Sí	No
Cambiar estado, responsable y evidencia	Sí, propuesto	Sí	No
Crear o editar proveedores	Sí	No	No
Consultar proveedores e informe	Sí	Sí	Sí

Esta matriz concreta el reparto del enunciado; documentad el acceso de ADMIN a tareas técnicas y quién importa o genera informes. Gestionar usuarios desde una pantalla completa puede esperar si no lo pide el profesor.

Ejemplo: ocultar el botón Guardar a VIEWER no es suficiente. Pablo debe mandar igualmente una petición de escritura con ese usuario. El backend la rechaza y los datos quedan igual. Referencia de apoyo: OWASP [R8].

7.3. Después el CRUD, en pequeñas piezas

1. Crear los DTO: los campos que la operación acepta y devuelve.
2. Crear la entidad: cómo se relaciona el dato con la tabla.
3. Crear el repositorio: acceso a los datos.
4. Crear el servicio: reglas, validación y operaciones que deben guardarse juntas.
5. Crear el controlador: recibe la petición y devuelve la respuesta.
6. Probar un alta y una consulta antes de añadir edición y filtros.

No pongáis toda la lógica en el controlador. Pensad que cambiar de pantalla no debería cambiar las reglas de una vulnerabilidad.

7.4. Operaciones que tienen que existir

Operación	Ruta propuesta	Qué debe hacer
Login	`POST /api/auth/login`	Comprobar credenciales
Consultar y crear activos	`GET /api/assets` y `POST /api/assets`	Listar y registrar
Editar o desactivar activo	`PATCH /api/assets/{id}`	Modificar un activo existente
Consultar vulnerabilidades	`GET /api/vulnerabilities`	Filtrar, ordenar y paginar
Ver detalle	`GET /api/vulnerabilities/{id}`	Mostrar datos, activos, CVSS y evidencias
Registrar vulnerabilidad	`POST /api/vulnerabilities`	Crear con estado Abierta
Editar vulnerabilidad	`PATCH /api/vulnerabilities/{id}`	Cambiar datos, responsable o fecha con control de versión
Cambiar estado	`PATCH /api/vulnerabilities/{id}/status`	Validar transición y evidencia
Guardar valoración	`POST /api/vulnerabilities/{id}/assessment`	Validar y conservar CVSS
Dashboard	`GET /api/dashboard/summary`	Totales por severidad y estado
Proveedores	`GET /api/providers`, `POST /api/providers`, `PATCH /api/providers/{id}`	Consultar, dar de alta y editar
Informe de proveedor	`GET /api/providers/{id}/report`	Mostrar sus pendientes

Las rutas de edición adicionales completan los casos de uso. Los nombres pueden cambiar si quedan documentados y ambos clientes los usan igual. Para importar y generar informes, añadid operaciones de lanzamiento y consulta del trabajo; se explican en los apartados 9 y 12.

7.5. Respuestas y filtros sin complicarse

Usad 200 para consulta o modificación correcta, 201 para un alta y 202 para una tarea aceptada. Un 400 significa datos inválidos; 401, falta de autenticación; 403, falta de permiso; 404, recurso que no existe; 409, duplicado o conflicto de edición.

El listado debe combinar severidad, estado, activo, responsable y mínimo/máximo de puntuación. Definid orden, número de página y tamaño máximo. Si el mínimo es mayor que el máximo, explicad el error; no devolváis resultados extraños.

Lo comprobáis así: con ANALYST creáis un activo, una vulnerabilidad asociada y consultáis el detalle; con VIEWER intentáis modificarla y comprobáis el rechazo. Guardad ejemplos de peticiones y respuestas en `docs/api/` sin credenciales reutilizables.

Los tres debéis responder: ¿qué hace el controlador?, ¿dónde se comprueba el permiso?, ¿qué diferencia hay entre 401 y 403?, ¿por qué el móvil y la web necesitan acordar los mismos campos?

8. Cuarto paso: guardar bien la gravedad CVSS

T07: lleva Alex, revisa Adri y demuestra Pablo.

8.1. Qué es y para qué sirve

CVSS permite expresar la gravedad técnica de una vulnerabilidad con una puntuación de 0 a 10. El vector es el texto que describe las métricas usadas para obtenerla. Aquí solo trabajáis con la parte **Base**.

No os piden memorizar la fórmula ni programarla desde cero. Sí comprobar que versión, vector y puntuación corresponden y conservarlos correctamente. La severidad se obtiene del número; no se elige un color al azar.

Score Base	Severidad
0,0	Ninguna
0,1-3,9	Baja
4,0-6,9	Media
7,0-8,9	Alta
9,0-10,0	Crítica

8.2. Qué hacéis, en orden

1. Acordar que la primera versión soporta CVSS 3.1 Base, si seguís la propuesta.
2. Preparar ejemplos contrastados con la calculadora oficial FIRST [R1].
3. Guardar versión, vector, score, severidad y fecha de valoración.
4. Comprobar que el vector empieza con la versión correcta y que contiene métricas Base válidas, completas y no repetidas.
5. Comprobar score numérico entre 0 y 10, con la precisión acordada.
6. Usar una biblioteca compatible, verificada con ejemplos FIRST, para comprobar correspondencia entre vector y score. La biblioteca evita escribir la fórmula desde cero.
7. Calcular la etiqueta de severidad en el backend. Si el cliente manda una etiqueta incompatible, rechazarla en vez de guardarla sin revisar.
8. Reutilizar la misma validación al crear desde formulario y al importar JSON.

Importante: comprobar solamente que empieza por `CVSS:3.1` y que el número está entre 0 y 10 no demuestra que la puntuación corresponda a ese vector. Si copiáis valores manualmente de FIRST, conservad las referencias, pero resolved también la validación de coherencia en el sistema.

8.3. Ejemplo y comprobación

Ejemplo del enunciado: vector `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`; puntuación 9,8; severidad Crítica.

Probad también versión 4.0 con vector 3.1, puntuación negativa, valor mayor de 10, métrica ausente y severidad Baja con score 9,8. Deben rechazarse. Revisad las fronteras 3,9/4,0; 6,9/7,0 y 8,9/9,0. Para probar coherencia, utilizad vectores con puntuaciones realmente verificadas; no inventéis un vector para cada frontera.

La gravedad técnica no es lo mismo que la urgencia empresarial: un activo expuesto o una fecha vencida pueden influir en el orden de trabajo, pero no justifican cambiar el CVSS guardado.

Los tres debéis responder: ¿qué es el vector?, ¿cómo sale la severidad?, ¿qué datos inválidos rechazáis?, ¿por qué no hay que programar la fórmula completa?

9. Quinto paso: importar veinte registros sin dejar datos a medias

T08: lleva Adri, revisa Pablo y demuestra Alex.

9.1. Qué queréis conseguir

Leer un fichero JSON con al menos veinte vulnerabilidades y guardarlas correctamente. Si una falla, no deben quedar guardadas solo las primeras. Es como una compra con varios artículos: o se confirma completa o se deshace.

9.2. Qué archivos preparáis

- `data/providers.json`: proveedores de prueba con servicio, criticidad, contrato y revisión.
- `data/assets.json`: activos y sus proveedores.
- `data/vulnerabilities_seed.json`: al menos veinte vulnerabilidades con identificador, título, descripción, fecha, activos, versión CVSS, vector, score, severidad y estado.
- `data/vulnerabilities_invalid.json`: un lote parecido con errores intencionados para la prueba.

Probad varias severidades, estados y activos. Incluid una vulnerabilidad que afecte a dos activos del mismo proveedor y otra a activos de distintos proveedores. Si inventáis el problema, usad un identificador interno como VFU-0001; no lo presentéis como un CVE real.

9.3. Qué hacéis, en orden

1. Leer el archivo y comprobar que tiene la estructura esperada y un tamaño admitido.
2. Validar campos, CVSS, referencias a activos y duplicados.
3. Si hay errores, indicar registro y campo: por ejemplo «Registro 12: versión y vector no coinciden».
4. Si los datos son válidos, iniciar una transacción para guardar el lote entero con relaciones y valoraciones.
5. Si ocurre un fallo durante la escritura, deshacer la transacción y marcar la importación como fallida.
6. Si termina bien, informar de cuántos registros se incorporaron.
7. Decidir qué ocurre al importar dos veces el mismo fichero. Propuesta sencilla: rechazar duplicados y preparar una base de demostración reinicializable.

No carguéis veinte registros cada vez que arranca el backend si eso crea duplicados. La preparación inicial debe tener un comportamiento definido.

9.4. Cómo demostrarlo de verdad

Primero apuntad filas e identificadores existentes. Importad el lote inválido y comprobad que no aparece ninguna incorporación parcial. Después importad el válido y comprobáis que existen los veinte o más registros con sus relaciones.

Hay dos pruebas diferentes: **validación** es rechazar el lote antes de guardar; **rollback** es deshacerlo cuando ya había empezado a escribir. Preparad además una prueba aislada que fuerce un fallo de persistencia, como una restricción única. Así no llamáis rollback a algo que nunca llegó a escribir.

Detalle para quien programa: si la importación se ejecuta en otro hilo, la transacción tiene que abrirse dentro del trabajo de ese hilo. En Spring, llamad a un servicio transaccional gestionado, no confiéis en que la transacción de la petición inicial viaje al executor ni en una llamada interna que no pase por el proxy.

Los tres debéis responder: ¿qué pasa si falla el registro 12?, ¿dónde ve el usuario el error?, ¿qué diferencia hay entre validar y hacer rollback?, ¿cómo evitáis duplicados al repetir la carga?

10. Sexto paso: seguir la resolución del problema

T09: lleva Pablo, revisa Alex y demuestra Adri.

10.1. Qué significan los estados

- **Abierta:** el problema está registrado y sigue pendiente.
- **Pendiente de remediación:** ya se está organizando o realizando la corrección.
- **Resuelta:** se ha aplicado una solución y hay una evidencia.
- **Cerrada:** se ha dado por finalizado el seguimiento según el procedimiento acordado.

El enunciado exige esos cuatro estados. Como propuesta, usad la secuencia Abierta > Pendiente > Resuelta > Cerrada. Exigir responsable y fecha para pasar a pendiente, y evidencia para resolver, hace el recorrido más claro. Estas condiciones concretas son reglas propuestas; documentad las que elijáis.

10.2. Qué hacéis, en orden

1. Escribir las transiciones permitidas antes de crear botones.
2. Añadir responsable y fecha objetivo a la vulnerabilidad.
3. Crear comentarios con autor y fecha, incluyendo evidencia breve como texto o referencia.
4. Implementar el cambio de estado y rechazar una transición no permitida.
5. Guardar en auditoría usuario, fecha, cambio anterior y nuevo.
6. Confirmar estado, comentario y auditoría juntos: si falla la operación, no dejar una evidencia que parezca confirmar un cambio inexistente.
7. Mostrar el resultado en el detalle de web y móvil.

Ejemplo: VFU-0001 pasa a Pendiente con un responsable y fecha; después alguien escribe «corrección aplicada y comprobada en entorno de pruebas» y la marca Resuelta. Un refresco desde Android muestra el nuevo estado.

Lo comprobáis así: Adri hace el recorrido sin ayuda de Pablo. Alex intenta saltar un paso que vuestra regla prohíbe y comprueba el error. Los tres identifican dónde aparece la evidencia y quién realizó el cambio.

Pregunta clave: ¿qué diferencia hay entre comentar «está arreglado» y guardar una transición válida con evidencia y auditoría?

11. Séptimo paso: que dos usuarios no se pisen los cambios

T10: lleva Alex, revisa Pablo y demuestra Adri.

11.1. El problema explicado con un ejemplo

Alex y Adri abren VFU-0001. Los dos ven la versión 7. Alex cambia el responsable y guarda. Adri cambia la fecha usando su copia antigua. Si el sistema acepta todo sin comprobar nada, la segunda escritura puede borrar el cambio de Alex.

No basta con decir «la base de datos es segura» o «usamos transacciones». Tenéis que enseñar este problema y la protección elegida.

11.2. Solución propuesta: número de versión

Cada registro tiene una versión de edición. Al guardar, el cliente envía la versión que había leído. El backend la compara con la actual. Si ya cambió, rechaza la edición con un conflicto y pide refrescar. En JPA se puede apoyar con `@Version`.

La versión del cliente importa: si el backend carga la fila más reciente pero nunca comprueba con qué versión estaba trabajando la persona, puede aceptar una edición antigua como si fuera nueva.

11.3. Qué hacéis para la prueba

1. Preparar una vulnerabilidad conocida en una base de pruebas.
2. Hacer que dos tareas lean la misma versión y esperen en un punto común antes de escribir. Ese punto se puede programar con una barrera; no dependáis solo de pulsar dos botones rápido.
3. En una prueba aislada sin protección, mostrar cómo se pierde un cambio. No dejar una opción insegura en el backend normal.
4. Repetir con protección. Una escritura se confirma y la otra recibe un conflicto, por ejemplo HTTP 409.
5. Mostrar que el cambio aceptado no se ha perdido.
6. Refrescar el segundo cliente, revisar el dato actual y repetir conscientemente el segundo cambio.
7. Comprobar que ambos cambios compatibles terminan conservados.

Mensaje comprensible: «Otra persona ha modificado este registro. Actualiza y revisa tu cambio». No anunciéis éxito si en realidad se rechazó una de las peticiones. Tampoco reintentéis automáticamente una transición sin revisar el estado nuevo.

Evidencia: resultado sin protección, resultado protegido, versiones y estado final. La documentación PostgreSQL [R6] sirve de apoyo para entender que no todos los niveles de aislamiento resuelven los mismos problemas.

Los tres debéis responder sin ayuda: ¿qué dato se perdía?, ¿cómo forzasteis que ocurriera?, ¿qué significa el conflicto?, ¿qué hace la persona para guardar el segundo cambio?

12. Octavo paso: generar un informe sin bloquear la aplicación

T11: lleva Adri, revisa Alex y demuestra Pablo.

12.1. Dos conceptos que hay que distinguir

Trabajo en segundo plano: el backend continúa una tarea mientras el usuario puede seguir usando la aplicación. Se puede realizar con hilos o un ExecutorService, que organiza un conjunto limitado de trabajadores.

Proceso externo: otro programa arrancado por el backend con ProcessBuilder. Tiene su propia ejecución y devuelve una salida y un código de finalización. Ejecutar un método en otro hilo no es lanzar un proceso externo.

12.2. Propuesta sencilla para cumplir ambos requisitos

El usuario pide un informe. El backend crea un trabajo con identificador. Un trabajador recoge los datos y lanza un pequeño programa Java separado que escribe un CSV. Al terminar, se ofrece la descarga. Así evitáis exigir otro lenguaje en el ordenador del profesor.

12.3. Qué hacéis, en orden

1. Decidir columnas: identificador, título, activos, proveedor, estado, severidad, score, responsable y fecha objetivo.
2. Crear un seguimiento del trabajo con estados En cola, En ejecución, Completado y Fallido.
3. Añadir un endpoint de lanzamiento, por ejemplo `POST /api/reports`, y otro de consulta, como `GET /api/jobs/{id}`.
4. Preparar datos de entrada controlados por el backend para el programa hijo.
5. Iniciarlo con ProcessBuilder, ejecutable y argumentos separados; no construir una orden libre con texto del usuario.
6. Recoger salida normal y errores sin que el proceso se quede bloqueado por no leerlos.
7. Establecer un tiempo máximo y gestionar el fallo si se supera.
8. Comprobar código de retorno, existencia y contenido del CSV.
9. Marcar el resultado y permitir descargar solo a quien esté autorizado.
10. Explicar qué ocurre si el backend se reinicia durante una tarea; no prometer recuperación que no habéis implementado.

Limitar hilos y cola evita lanzar trabajos sin control. Escapad correctamente comillas y separadores del CSV y tratad texto que una hoja de cálculo pueda interpretar como fórmula. Si generáis HTML, escapad su contenido. El cliente no elige cualquier ruta del disco del servidor.

Lo comprobáis así: Pablo pide un informe y lo abre. Alex fuerza un fallo controlado, por ejemplo ejecutable de pruebas ausente o timeout, y comprueba que aparece un error comprensible, no una espera infinita. Guardad inicio del proceso, código de salida y archivo final.

Los tres debéis responder: ¿qué diferencia hay entre hilo y proceso?, ¿qué significa el código de retorno?, ¿cómo se entera el usuario de que ha terminado?, ¿qué pasa si falla?

13. Noveno paso: relacionar proveedores con sus riesgos

T12: lleva Pablo, revisa Adri y demuestra Alex.

13.1. Qué queréis conseguir

Que se pueda dar de alta un proveedor, saber qué activos presta y consultar las vulnerabilidades pendientes que le afectan. Es el bloque de Sistemas de Gestión Empresarial, integrado en el mismo producto.

13.2. Qué hacéis, en orden

1. Dibujar el proceso de negocio: alta > evaluación de criticidad > contrato > relación con activos > revisión > acción si hay riesgo.
2. Representarlo como BPMN o el formato acordado. Es un proceso de decisiones y responsables, no otro dibujo de tablas.
3. Guardar servicio, criticidad, contrato o referencia contractual, revisión y contacto ficticio.
4. Permitir alta y edición a ADMIN. ANALYST y VIEWER consultan.
5. Vincular proveedores con activos y cargar ejemplos junto a la preparación inicial.
6. Implementar una regla de riesgo documentada.
7. Crear el informe con pendientes, severidad y fecha objetivo, sin duplicar vulnerabilidades por sus activos.

13.3. Regla propuesta explicada

Si HostingSur tiene criticidad alta y alguno de sus activos tiene una vulnerabilidad crítica Abierta o Pendiente, se muestra «En riesgo». Si ha vencido su fecha de revisión, aparece además «Revisión pendiente». Dejad escrito si las condiciones se combinan con «y» o con «o».

En esta propuesta, cuando VFU-0001 pasa a Resuelta deja de contarse como pendiente técnica aunque todavía no esté Cerrada. Si elegís otra definición, cambiadla de forma consistente en todos los informes.

Calculad el indicador desde los datos actuales o definid cuándo se actualiza. Si guardáis una casilla de riesgo y nunca la recalculáis, puede seguir roja después de resolver el problema.

Lo comprobáis así: Alex crea un proveedor como ADMIN, lo vincula a un activo y abre el informe. Adri comprueba un proveedor sin activos, otro con revisión vencida y una vulnerabilidad compartida por dos activos. Intentar editar como ANALYST debe fallar.

Los tres debéis responder: ¿por qué aparece En riesgo?, ¿qué cambia al resolver una vulnerabilidad?, ¿por qué no se suman dos veces los problemas compartidos?

14. Décimo paso: construir la web que usará la persona

T13: lleva Alex, revisa Adri y demuestra Pablo. T14: lleva Adri, revisa Pablo y demuestra Alex.

14.1. Antes de programar: dibujar el recorrido

Haced wireframes: dibujos sencillos de las pantallas y de cómo se pasa de una a otra. No hace falta que sean bonitos. Deben permitir contestar: «¿Dónde entro? ¿Cómo creo un activo? ¿Cómo registro una vulnerabilidad? ¿Dónde cambio su estado?».

14.2. Pantallas y orden de construcción

Pantalla	Qué debe poder hacer la persona	Cuándo está lista
Login	Entrar y entender un fallo de credenciales	Usa autenticación real
Activos	Consultar, crear, editar y desactivar	Los cambios quedan guardados
Vulnerabilidades	Buscar, combinar filtros y paginar	Las peticiones usan filtros reales
Alta y edición	Rellenar datos, activos y CVSS	Muestra errores por campo
Detalle	Ver CVSS, responsables, fechas y evidencias	Se puede seguir la remediación
Dashboard	Ver cantidades y seleccionar severidad o estado	Lleva a datos filtrados y ofrece alternativa textual
Proveedores	Consultar; ADMIN crea y edita	Respeta permisos y muestra el informe
Importaciones e informes	Lanzar, ver estado y abrir resultado	Diferencia carga, error y éxito

14.3. Qué hacéis, en orden

1. Crear componentes reutilizables: botón, campo, selector, tabla y aviso. Guardar una guía de estilo básica con colores, tipografía y ejemplos de componentes.
2. Preparar un servicio HTTP común con URL de API y gestión de autenticación.
3. Implementar login y un listado real antes de los gráficos.
4. Completar alta de activo y vulnerabilidad, detalle y cambio de estado.
5. Añadir filtros combinados, orden y paginación.
6. Añadir resumen y gráficos básicos interactivos con tabla o texto equivalente.
7. Completar proveedores, tareas e informes.
8. Preparar exportación CSV y resolver con el profesor la duda del PDF.
9. Revisar accesibilidad y errores en todas las pantallas.

Separad la parte que dibuja la pantalla de la que gestiona el estado y hace peticiones. No copiéis el mismo código de conexión en diez componentes. Los datos simulados ayudan a empezar una pantalla, pero al cerrarla debe usar el backend real.

14.4. Lo que suele faltar al principio

Cada pantalla necesita estado de carga, ausencia de datos, error y reintento. Una lista sin resultados no significa lo mismo que un fallo del servidor. Si guardar devuelve conflicto, no borres el formulario ni muestres «todo correcto».

Al exportar, indicad qué contiene el CSV. Propuesta: todos los registros de la selección filtrada que el usuario puede consultar, no solo los que caben en la página visible.

14.5. Accesibilidad explicada con ejemplos

El enunciado pide WCAG 2.1 AA. Navegad sin ratón: debéis poder llegar a campos y botones y ver dónde está el foco. Poned etiquetas reales a los campos y errores asociados. «Crítica» tiene que leerse, no depender solo de un color rojo. Un gráfico necesita también los números en texto o tabla.

Comprobad contraste, estructura de títulos, funcionamiento de diálogos y lectura de mensajes. Las herramientas automáticas ayudan, pero no sustituyen las pruebas manuales ni demuestran por sí solas conformidad completa. Guardad hallazgos y correcciones con referencia a W3C [R7].

Lo comprobáis así: Pablo completa el recorrido de crear activo > crear vulnerabilidad > asignar > resolver usando las pantallas de Alex. Alex demuestra los filtros y el dashboard de Adri. Los tres revisan una operación prohibida y una entrada incorrecta.

Pregunta clave: ¿cómo sabemos que el dato se ha guardado de verdad y no solo ha cambiado lo que aparece en pantalla?

15. Undécimo paso: consultar y actualizar desde Android

T15: lleva Pablo, revisa Alex y demuestra Adri. T16: lleva Alex, revisa Pablo y demuestra Adri.

15.1. Qué queréis conseguir

Entrar desde Android, ver las mismas vulnerabilidades de la web, filtrar y consultar su detalle. ANALYST podrá realizar una actualización limitada, como cambiar estado o responsable. Además, el móvil conservará una copia de lo último consultado para leer sin conexión.

15.2. Qué hacéis, en orden

1. Configurar URL de API y permiso de Internet.
2. Crear login contra el backend real.
3. Crear el cliente de red y el repositorio que obtiene los datos.
4. Preparar lista, filtros y detalle con versión, vector, score y severidad.
5. Usar ViewModel para gestionar el estado de pantalla y su ciclo de vida.

6. Mostrar carga, error, lista vacía y opción de reintentar.
7. Añadir cambio permitido para ANALYST con el control de versión del registro.
8. Guardar con Room los datos consultados y la fecha de actualización.
9. Mostrar esos datos sin conexión e indicar que pueden estar desactualizados.
10. Probar cierre de sesión, token caducado y limpieza de caché al cambiar de usuario.

La pantalla es lo que se ve. ViewModel conserva y prepara su estado. El repositorio decide de dónde vienen los datos: red o copia local. Room sirve para guardar registros estructurados, no solo preferencias. La guía Android [R5] amplía estas ideas.

15.3. La URL del emulador: el fallo típico

`localhost` dentro del emulador apunta al emulador, no al ordenador. En el emulador oficial, `10.0.2.2` permite acceder al loopback del ordenador. Con el puerto propuesto usaríais `http://10.0.2.2:8080/api/` [R9].

En un móvil real, usar la IP local accesible del ordenador y revisar red, interfaz de escucha y firewall. Si usáis HTTP local, limitad la excepción de tráfico sin cifrar a depuración. Para un servicio expuesto, HTTPS. CORS afecta al navegador; tocarlo no suele resolver por sí solo una conexión Android fallida.

15.4. Qué significa aquí sin conexión

Solo pedimos lectura de la copia guardada. No necesitáis permitir cambios offline y sincronizarlos después. Indicad «Sin conexión» y la fecha del dato; desactivad escrituras que requieren red.

Si solo guardáis páginas visitadas, no digáis que todo el catálogo está disponible. Si filtráis sin conexión, aclarad que buscáis dentro de lo guardado. Una respuesta de error no debe borrar la última lista válida como si fuera un resultado vacío.

Lo comprobáis así: Adri abre VFU-0001 en el móvil; Pablo cambia su estado desde la web; Adri refresca y lo ve. Después carga datos, desconecta, reabre y consulta la copia. Finalmente prueba un cambio desde móvil y lo comprueba desde web.

Los tres debéis responder: ¿por qué Android ve los mismos datos?, ¿qué ocurre sin conexión?, ¿qué hace Room?, ¿por qué no sirve localhost para llegar al ordenador?

16. Duodécimo paso: empaquetar, integrar y probar todo

T17: lleva Adri, revisa Alex y demuestra Pablo. T18: lleva Pablo, revisa Adri y demuestra Alex.

16.1. Preparar algo que otro pueda ejecutar

Para DI, si seguís la opción web, compilad la SPA y servidla con Nginx en Docker. Preparad Dockerfile y `docker-compose.yml`. Ejecutar solo el servidor de desarrollo no demuestra la distribución final pedida.

Comprobad que una ruta interna de la SPA funciona al recargar directamente. Si Nginx actúa de proxy hacia la API, la web puede usar `/api`. El nombre `backend` de un contenedor se resuelve dentro de la red Docker; no asumáis que el navegador del profesor puede abrirlo como dirección pública.

Para Android, generad un APK de prueba y anotad cómo instalarlo. `.gitignore` excluye APK y carpetas de construcción: verificad que el archivo o su enlace de entrega sea realmente accesible al profesor. Conservar solo el proyecto no equivale a haber entregado el APK.

16.2. Pruebas mínimas que debéis repetir

Prueba	Qué hacéis	Qué tendría que pasar
P01. Arranque limpio	Seguir instrucciones en una copia limpia	Se construye y arranca sin archivos privados
P02. Login y rol	Probar credenciales y escritura de VIEWER	Se acepta o rechaza según corresponda
P03. Activos	Crear, editar y desactivar	La base de datos conserva los cambios
P04. Relaciones	Asociar una vulnerabilidad a dos activos	Ambos vínculos existen sin duplicados
P05. CVSS	Probar ejemplos correctos y errores	Se guarda solo una valoración coherente
P06. Importación válida	Cargar veinte o más vulnerabilidades	Se guarda el lote completo
P07. Validación y rollback	Rechazo previo y fallo durante escritura	No quedan cambios parciales
P08. Filtros	Combinar los filtros pedidos	Solo aparecen las coincidencias
P09. Remediación	Cambiar responsable, fecha, evidencia y estado	Se aplica la regla y queda auditoría
P10. Concurrencia	Ejecutar la carrera y la protección	No hay pérdida silenciosa; se recupera el cambio
P11. Proceso	Generar CSV y provocar un fallo controlado	Archivo válido o fallo informado
P12. Proveedores	Ver regla, revisión y activos compartidos	Totales correctos y permisos respetados
P13. Web y móvil	Cambiar en uno y refrescar el otro	La información coincide
P14. Caché	Desconectar después de consultar	Datos locales con aviso de antigüedad
P15. Accesibilidad	Teclado, errores, contraste y gráficos	Recorrido utilizable y correcciones documentadas
P16. Distribución	SPA empaquetada y APK de prueba	Otra persona los ejecuta siguiendo instrucciones

Estas son pruebas del proyecto, no dieciséis aplicaciones diferentes. Automatizad sobre todo reglas CVSS, permisos, importación, integridad y concurrencia; completad con guiones reproducibles para pantallas, Android y demostración.

16.3. Cómo guardar una prueba útil

Para cada prueba escribid objetivo, datos de partida, pasos, resultado esperado, resultado observado y archivo de evidencia. Ejemplo: «P07: importar lote con error de persistencia; esperado: ninguna incorporación; observado: mismos identificadores antes y después; evidencia: salida de prueba y consulta SQL».

Usad `docs/pruebas/` y `docs/evidencias/`. No guardéis contraseñas, tokens completos ni datos reales en capturas o logs. Probádlo contra la base de datos final de pruebas; que funcione en una base alternativa no demuestra todos los comportamientos de PostgreSQL.

Trabajo compartido: Alex ejecuta el recorrido completo que preparó Pablo sin pedirle cada paso. Adri anota las instrucciones que falten. Después cada uno repite al menos una prueba de una parte que no lideró. Si solo el autor puede arrancarla, falta trabajo de entrega.

17. Los trabajos de cada módulo, explicados por entregable

El programa no es toda la entrega. También debéis explicar su utilidad, documentarlo en inglés y medir mejoras. Usad los responsables del apartado 4.4, manteniendo las revisiones y las aportaciones de todos.

17.1. Ampliación de Inglés: documentación escrita

Qué entregáis: README en inglés, glosario bilingüe con al menos 30 términos y executive summary de una página. Preparad también recursos de interfaz para inglés y documentación de API en inglés según tareas e indicadores del enunciado.

Cómo hacerlo: traducir y revisar las instrucciones reales de instalación; seleccionar palabras que utilizáis en el proyecto; escribir término inglés, traducción, definición y un ejemplo. Para el resumen ejecutivo, contad al responsable de riesgos qué problemas hay, qué pendientes son prioritarios y qué acción proponéis. Las cifras deben salir de vuestros datos de demostración.

Comprobación entre los tres: cada uno explica cinco palabras que escribió otro. Revisad textos asistidos por IA y dejad constancia de la ayuda; no entreguéis frases que ninguno pueda explicar.

17.2. Inglés Profesional: hablar y demostrar

Qué entregáis: vídeo de 3-5 minutos en inglés, con los tres participando, su guion y guion/diapositivas del bloque oral. La defensa incluye al menos tres minutos en inglés y preguntas relacionadas.

Cómo hacerlo: reparto inicial del vídeo: Alex presenta el problema, Adri enseña un recorrido web y Pablo muestra Android y el valor del resultado. En los ensayos cambiad también de parte para no memorizar un único texto.

Preparad frases claras para explicar API compartida, CVSS, permisos y caché. No hace falta recitar un texto complicado. Probad audio, duración y enlaces antes de entregar. El vídeo grabado no sustituye la demostración en directo.

17.3. IPE II: organización y viabilidad

Qué entregáis: Lean Canvas, comparación de al menos tres soluciones reales, DAFO, forma jurídica justificada, presupuesto en hoja de cálculo, plan de trabajo, al menos dos actas, contribuciones y portfolios actualizados.

Cómo hacerlo: el Canvas responde quién tiene el problema, qué ofrecéis y cómo se sostendría. El DAFO separa fortalezas/debilidades internas y oportunidades/amenazas externas. Para competencia, consultar fuentes oficiales actuales y anotar fecha, público, funciones y precio publicado o ausencia de precio; no inventarlo.

En el presupuesto, multiplicad horas por tarifa de cada perfil y sumad infraestructura y licencias. Separad coste y precio de venta. Si calculáis margen sobre ventas, precio = coste / (1 - margen); no es igual que sumar un porcentaje sobre coste. Indicad qué cifras son hipótesis académicas.

La forma jurídica se justifica con fuentes oficiales actuales, no copiando una preferencia sin revisar responsabilidad, socios y costes. Cada uno actualiza su CV, GitHub y LinkedIn con su aportación real. Las actas pueden aprovechar revisiones del equipo si registran fecha, asistentes, decisión y tareas.

17.4. Digitalización: qué mejora en la empresa

Qué entregáis: informe de 4-6 páginas y diagrama de flujo de datos, con diagnóstico antes/después, tecnologías, riesgos y al menos tres acciones de hoja de ruta.

Cómo hacerlo: imaginad que antes la empresa usaba hojas dispersas y correos. Explicad qué cambia al centralizar activos, problemas y responsables. Dibujad dónde nace el dato, quién lo valida, dónde se guarda y quién lo usa para decidir.

Proponed tres mejoras futuras con plazo y motivo, por ejemplo importación periódica, resúmenes para responsables y revisión de proveedores. Distinguid lo que habéis implementado de lo que proponéis para después. Si citáis IA futura, no digáis que ya forma parte del programa.

17.5. Sostenibilidad: medir dos mejoras de verdad

Qué entregáis: informe de 3-5 páginas, relación con ODS 8, 9, 12 y 16, impactos del ciclo de vida, dos mejoras con medición antes/después y un decálogo de diez prácticas.

Mejora propuesta 1: paginación. Medid cuánto ocupa y tarda la carga inicial del listado completo frente a una primera página, usando los mismos datos. Aclarad que comparáis carga inicial; descargar después todas las páginas no es el mismo escenario.

Mejora propuesta 2: caché Android. Repetid un recorrido de consulta y contad peticiones con y sin reutilizar datos locales. Mismo dispositivo, recorrido y condiciones; registrar varias repeticiones y usar, por ejemplo, la mediana. El porcentaje de reducción es (antes - después) / antes × 100 si el valor inicial no es cero.

No copiéis como propios los números de ejemplo del enunciado. Reducir bytes o peticiones es un indicador útil, pero no permite afirmar una cantidad de CO2 ahorrada sin un método que lo demuestre.

Comprobación compartida: Pablo repite una de las mediciones de Adri; Alex explica qué se ha medido y qué no. Los tres deben poder defender si la comparación es justa.

18. Preparar una memoria única que los tres entendáis

18.1. Qué es la memoria

Es el documento final donde explicáis qué habéis hecho, por qué lo hicisteis así y cómo lo comprobasteis. No es pegar todos los README uno detrás de otro ni una colección de capturas sin explicación.

OBLIGATORIO: un único PDF con partes técnicas y transversales, reparto, contribuciones individuales y declaración de uso de IA. Se entrega completo en cada módulo, junto con el enlace al repositorio accesible al profesorado.

18.2. Índice propuesto

1. Portada, autores e índice.
2. Problema, objetivos y alcance.
3. Plan, reparto y contribuciones de Alex, Adri y Pablo.
4. Arquitectura y decisiones, con alternativas descartadas.
5. Modelo de datos e importación.
6. API, autenticación, permisos y modelo breve de amenazas.
7. Concurrencia, segundo plano y proceso externo.
8. Web: diseño, componentes, accesibilidad y distribución.
9. Android: conexión, estado de pantalla y caché.
10. Proveedores: proceso, regla e informe.
11. Pruebas, resultados, evidencias y limitaciones.
12. Inglés escrito y oral.
13. IPE, digitalización y sostenibilidad.
14. Conclusiones, aprendizaje y mejoras futuras.
15. Declaración de IA, referencias y anexos.

18.3. Una forma fácil de redactar cada apartado

Responded cuatro preguntas: **¿qué necesitábamos?, ¿qué hicimos?, ¿por qué así?, ¿cómo sabemos que funciona?**

Ejemplo: «Necesitábamos impedir que dos ediciones se sobrescribieran. Añadimos control de versión. Lo elegimos para detectar una copia antigua antes de aceptar el cambio. La prueba P10 fuerza dos escrituras y comprueba el conflicto y la recuperación».

Cada captura lleva título, prueba o requisito que demuestra y explicación breve. No metáis el código entero: citad archivos y mostrad solo fragmentos relevantes. No escribáis como implementada una función que solo está prevista.

18.4. Seguridad y uso de IA, con ejemplos propios

El modelo de amenazas explica qué protegéis, de quién y con qué controles. Ejemplos: impedir que VIEWER cambie datos; rechazar un lote inválido; no perder cambios; limitar entradas del proceso de informe; no filtrar tokens por logs. Relacionad cada control con una prueba.

La declaración de IA debe indicar herramienta, tarea asistida, partes afectadas, revisión y verificación. Esta guía es apoyo para planificar, no prueba de que el proyecto esté construido. Cada uno debe entender el código y los textos que presenta.

Revisión compartida: Alex revisa un apartado técnico de Pablo, Adri uno de Alex y Pablo uno de Adri; después cambiad de lector en la revisión final. Si alguien no puede explicar un párrafo, se aclara y reescribe antes de entregar.

19. Prepararos para preguntas a cualquiera de los tres

19.1. Lo que cuenta en la nota

El enunciado reparte 30 % a memoria, 20 % a archivos/código y 50 % a defensa. Cada profesor evalúa su módulo con sus evidencias. La presentación dura 20 minutos y hay 5 minutos adicionales de preguntas.

Por eso no conviene que Alex sepa solo login, Adri solo gráficos y Pablo solo Android. Cualquiera debe poder seguir un dato por todo el sistema y defender las pruebas importantes.

19.2. Guion orientativo de veinte minutos

Tiempo	Qué enseñáis	Primera voz propuesta
0:00-2:00	Problema, arquitectura, arranque y salud	Alex
2:00-4:00	Login, VIEWER/ANALYST y operación prohibida	Adri
4:00-6:00	Importación válida, fallo y rollback	Pablo
6:00-9:00	Alta de activo/vulnerabilidad, CVSS, asignación y evidencia	Alex
9:00-11:00	Android, actualización compartida y caché	Adri
11:00-13:00	Proveedor, vínculo, regla e informe del proceso hijo	Pablo
13:00-15:00	Carrera sin protección y solución	Pablo introduce; Alex y Adri explican el resultado
15:00-18:30	Bloque en inglés: producto, decisiones y resultados	Los tres, con tiempos semejantes
18:30-20:00	Viabilidad, digitalización, sostenibilidad y conclusión	Los tres, con tiempos semejantes

Esta primera distribución busca intervenciones cercanas a 6-7 minutos por persona, contando los bloques compartidos. Medid los tiempos reales. Los papeles de la defensa no tienen por qué coincidir con quien

programó: Adri muestra Android aunque lo haya liderado Pablo, y Pablo presenta importación aunque la haya liderado Adri.

19.3. Tres ensayos diferentes

Ensayo 1 - Guiado: cada uno presenta el bloque previsto y los demás corrigen términos, pasos y tiempos. Todos tienen acceso a la lista de arranque.

Ensayo 2 - Intercambio: cambiáis presentador en backend, web y Android. Quien construyó la parte no ayuda durante los dos primeros minutos. Después se anotan dudas y se corrigen.

Ensayo 3 - Preguntas individuales: cada uno responde por separado a cinco preguntas de temas mezclados, sin que otro complete sus frases. Después comparáis respuestas y repetís las débiles. Esto os prepara si os preguntan a la vez o en una defensa individual.

La meta es contestar con palabras propias, no memorizar respuestas idénticas. Si una pregunta llega a uno, ese empieza y termina su explicación antes de que otro aporte un matiz.

19.4. Banco de preguntas que todos deben dominar

1. ¿Qué ocurre desde que pulsas Guardar hasta que el dato llega a la base de datos?
2. ¿Por qué web y Android ven la misma información?
3. ¿Dónde se impide que VIEWER escriba aunque cambie su pantalla?
4. ¿Cómo relacionáis varios activos con una vulnerabilidad?
5. ¿Qué diferencia hay entre CVSS, severidad y prioridad de trabajo?
6. ¿Cómo comprobáis que vector y puntuación son coherentes?
7. ¿Qué pasa si falla el registro 12 de un lote?
8. ¿Qué diferencia hay entre rechazar antes de escribir y hacer rollback?
9. ¿Cómo forzáis una carrera y qué dato se pierde sin protección?
10. ¿Qué significa el conflicto de versión y cómo se recupera el segundo cambio?
11. ¿Qué diferencia hay entre hilo y proceso hijo?
12. ¿Qué hace el backend si el proceso falla o tarda demasiado?
13. ¿Por qué HostingSur aparece en riesgo?
14. ¿Cómo evitáis contar dos veces una vulnerabilidad del mismo proveedor?
15. ¿Qué muestra Android sin conexión y qué no puede hacer?
16. ¿Cómo se instala el programa en el equipo del profesor?
17. ¿Qué dos mejoras de sostenibilidad medisteis y cómo repetisteis la medida?
18. ¿Qué parte necesitó más aprendizaje y cómo se repartieron las contribuciones?
19. ¿Qué os ayudó a hacer la IA y cómo comprobasteis el resultado?
20. ¿Qué cambiaríais con más tiempo y por qué?

Para cada respuesta intentad cubrir: función, recorrido de datos, decisión y prueba. Nadie necesita recordar de memoria la ruta exacta de todas las clases, pero sí localizar las principales y explicar su relación.

19.5. Preparación de la demo

Ensayad con un conjunto estable de datos y una forma segura de reinicializarlo. No repitáis la importación sobre la misma base sin tener resueltos los duplicados. Preparad pantallas y archivos y medid lo que tarda el proceso hijo.

Guardad un vídeo, capturas y logs como respaldo ante incidencias. Si los usáis, indicad que son una evidencia grabada; no los presentéis como ejecución en directo. La defensa completa debe incluir entrada inválida, operación no autorizada, consulta de evidencia, concurrencia y valor para el negocio.

20. Lista de entrega y vuestra primera sesión

20.1. Qué debe quedar en el repositorio y la entrega

Carpeta o archivo	Qué debe contener
`backend/`	API ejecutable, construcción, configuración de ejemplo y pruebas
`database/`	Esquema, semillas, consultas y explicación del modelo
`data/`	Activos, proveedores y JSON válido e inválido
`desktop-client/`	Web por componentes y configuración de distribución
`android-app/`	Proyecto y referencia accesible al APK de prueba
`docs/api/`	Contrato, ejemplos, errores y autenticación en inglés
`docs/equipo/`	Tablero, decisiones y matriz de conocimiento
`docs/pruebas/` y `docs/evidencias/`	Guiones, resultados, capturas y logs depurados
`docs/sge/`	Proceso BPMN, reglas e informe de proveedores
`docs/english/`	Glosario, resumen, guiones, diapositivas y vídeo/enlace
`docs/ipe/`	Canvas, competencia, DAFO, forma jurídica, presupuesto, actas y portfolios
`docs/digitalizacion/`	Informe de 4-6 páginas y flujo de datos
`docs/sostenibilidad/`	Informe de 3-5 páginas, dos medidas y decálogo
`docs/memoria/`	Memoria común, declaración de IA y anexos
`README.md`	Autores, requisitos, instalación, configuración, arranque, pruebas y usuarios de prueba; en inglés

20.2. Antes de entregar, marcad juntos

[ ] Las dudas académicas de DI e Inglés tienen criterio acordado o interpretación cubierta.

[ ] La API, la base de datos y ambos clientes se construyen y arrancan siguiendo instrucciones.

[ ] Los permisos se comprueban en backend y no hay contraseñas legibles ni secretos reales en la entrega.

[ ] Se pueden preparar datos, registrar activo/vulnerabilidad, remediar y consultar evidencias: CU1-CU5.

[ ] Hay veinte o más vulnerabilidades válidas y una prueba de importación errónea sin datos parciales.

[ ] CVSS, relaciones, filtros, dashboard y fechas son coherentes.

[ ] La carrera y su protección se muestran de forma reproducible.

[ ] Funcionan la tarea en segundo plano y el proceso externo con errores controlados.

[ ] Proveedores, vínculo, contrato, revisión, regla e informe están integrados.

[ ] La web está distribuida y revisada por accesibilidad; Android tiene APK y caché comprobada.

[ ] Los entregables de los diez módulos están presentes y sus cifras son reales o hipótesis identificadas.

[ ] El vídeo dura 3-5 minutos y participan los tres; el bloque oral en inglés dura al menos tres minutos.

[ ] Hay presupuesto, tres competidores, al menos dos actas y portfolios actualizados.

[ ] Las dos mejoras sostenibles tienen método, datos antes/después y límites explicados.

[ ] La memoria incluye contribuciones individuales, conclusiones y declaración de IA.

[ ] Alex, Adri y Pablo han demostrado partes que no lideraron y respondido preguntas por separado.

[ ] La defensa cabe en veinte minutos y tiene reparto equilibrado.

[ ] El profesorado puede abrir el repositorio y los archivos; se sube la memoria común y enlace en cada módulo.

20.3. Qué hacéis en vuestra primera sesión

1. Los tres leéis los apartados 1-4 y confirmáis el reparto inicial.
2. Alex abre T01 y registra tecnologías y dudas; Adri revisa la decisión y Pablo la resume sin leerla palabra por palabra.
3. Adri abre T02 y prepara los proyectos; Pablo comprueba herramientas y Alex sigue las instrucciones.
4. Pablo prepara el primer dibujo de tablas de T03; Alex revisa relaciones y Adri recorre dónde se guarda VFU-0001.
5. Dejad por escrito qué está hecho, qué está pendiente y qué necesita cada uno para la siguiente sesión.

Primer objetivo pequeño: que el backend arranque y que los tres sepáis explicar cómo una vulnerabilidad terminará guardada y visible en web y móvil. Todavía no necesitáis todos los gráficos ni todos los informes.

21. Fuentes y qué debéis contrastar

Fuente académica: `Proyecto_Intermodular_2DAM_VulnFollowUp (1).pdf`, 23 páginas. Alcance y casos de uso: páginas 3-6; responsabilidades y módulos: 7-18; entregables: 18-19; defensa y rúbrica: 19-23. Se revisaron los README para describir el punto de partida, no como prueba de funcionalidades implementadas.

Referencias técnicas oficiales consultadas para esta guía el 29 de octubre de 2026:

- [R1] FIRST, especificación y calculadora CVSS 3.1: https://www.first.org/cvss/v3.1/specification-document y https://www.first.org/cvss/calculator/3.1.
- [R2] Spring Boot, requisitos de sistema: https://docs.spring.io/spring-boot/system-requirements.html.
- [R3] Vite, inicio y requisitos: https://vite.dev/guide/.
- [R4] React, opciones para crear una aplicación: https://react.dev/learn/creating-a-react-app.
- [R5] Android, arquitectura con datos sin conexión: https://developer.android.com/topic/architecture/data-layer/offline-first.
- [R6] PostgreSQL, aislamiento de transacciones: https://www.postgresql.org/docs/current/transaction-iso.html.
- [R7] W3C, referencia de criterios WCAG: https://www.w3.org/WAI/WCAG21/quickref/.
- [R8] OWASP, seguridad de servicios REST: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html.
- [R9] Android, red del emulador: https://developer.android.com/studio/run/emulator-networking.

Comprobad las versiones concretas al crear los proyectos. La guía propone un camino y un reparto de equipo; no contiene resultados de pruebas todavía no realizadas ni certifica cumplimiento legal o accesibilidad completa. Las condiciones académicas definitivas las fija el profesorado.